electronics

Article

StatePre: A Large Language Model-Based State-Handling Method for Network Protocol Fuzzing

Yifan Zhang [1,2], Kailong Zhu [1,2], Jie Peng [1], Yuliang Lu [1,2,*], Qian Chen [1] and Zixiong Li [1,2]

[1] College of Electronic Engineering, National University of Defense Technology, Hefei 230037, China; zhangyifan.727@nudt.edu.cn (Y.Z.); zhukailong@nudt.edu.cn (K.Z.); pengjie22@nudt.edu.cn (J.P.); harutoo@126.com (Q.C.); lizixiong@nudt.edu.cn (Z.L.)
[2] Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation, Hefei 230037, China
* Correspondence: luyuliang@nudt.edu.cn

Abstract: As essential components for communication, network protocol programs are highly security-critical, making it crucial to identify their vulnerabilities. Fuzzing is one of the most popular software vulnerability discovery techniques, being highly efficient and having low false-positive rates. However, current network protocol fuzzing is hindered by the coarse-grained and missing state annotations in programs. The current solutions primarily rely on the manual modification of programs, which is inefficient and prone to omissions. In this paper, we propose StatePre, a novel state-handling method for stateful network protocol programs, which leverages large language model (LLM) code- and text-understanding capabilities to analyze request for comments (RFC)-defined state knowledge and optimize the state handling of programs for fuzzing. StatePre automatically refines coarse-grained state annotations and complements missing state annotations in programs to ensure precise state tracking and fuzzing effectiveness. We implement a prototype of StatePre. The evaluation shows that programs modified with StatePre, with fine-grained and comprehensive state annotations, achieve better fuzzing efficiency, higher code coverage, and improved crash detection compared to those not modified with StatePre. Moreover, StatePre demonstrates good scalability, thus is applicable to various network protocol programs.

Keywords: fuzzing; network protocol program; state handling; large language model

check for updates

Academic Editor: Sana Ullah Jan

Received: 9 April 2025
Revised: 30 April 2025
Accepted: 6 May 2025
Published: 9 May 2025

Citation: Zhang, Y.; Zhu, K.; Peng, J.; Lu, Y.; Chen, Q.; Li, Z. StatePre: A Large Language Model-Based State-Handling Method for Network Protocol Fuzzing. *Electronics* **2025**, *14*, 1931. https://doi.org/10.3390/electronics14101931

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Network protocols are the rules and conventions that govern the communication between different entities in a network, specifying the format, order, and error handling of data transmission to ensure network communication [1]. Network protocol programs are the implementations of network protocols, enabling network devices to interact based on certain rules. However, due to their exposure to remote attack surfaces, network protocol programs inherently possess higher security vulnerability risks compared to localized applications. When exploited by malicious actors, these vulnerabilities can serve as entry points for unauthorized network penetration, posing severe threats to cybersecurity [2]. An example is the EternalBlue exploit, targeting Microsoft's Server Message Block (SMB) protocol program [3], which precipitated the global WannaCry ransomware pandemic [4], encrypting data across millions of devices [5].

Fuzzing is extensively utilized in the security assessment of protocol implementations to uncover potential vulnerabilities [6]. Protocol fuzzing involves sending automatically

generated malformed data to protocol programs while monitoring their behavior (e.g., crashes or anomalies) to uncover vulnerabilities. In protocol fuzzing, the protocol state is a critical component due to the stateful nature of protocol programs. Existing methods, such as AFLNet [7] and StateAFL [8], primarily focus on runtime protocol state tracking within fuzzer designs. These methods demonstrate that proper state tracking can enhance the ability to detect vulnerabilities. However, the target programs inherently suffer from coarse-grained and missing state annotations—a consequence of developers' misinterpretations of protocol specifications and implementation oversights. To better leverage the carefully designed state-tracking mechanism of protocol fuzzers, thereby more effectively conducting fuzzing, it is also crucial to refine the target programs in the preprocessing stage.

Existing methods attempt to refine the target programs through state-handling techniques, yet lack dedicated optimizations. State handling—the explicit reconciliation between implementation-specific state annotations and RFC specifications—is critical to mitigate the hidden inconsistencies arising from ad hoc coding practices or ambiguous protocol interpretations. While implementation-level imperfections may not disrupt programs' normal operations, they critically hinder fuzzing effectiveness by distorting state-space representation and transition modeling.

In this paper, we focus on optimizing state handling. To design an effective and efficient state-handling method, three challenges have to be overcome. Challenge 1 is precision of state alignment. Achieving precise state alignment remains problematic due to divergent developer interpretations of RFC specifications, which often result in program-specific state deviations that coverage-guided fuzzing fails to detect, thereby overlooking critical vulnerabilities. Challenge 2 is the automation of annotation refinement. Most existing techniques rely heavily on human involvement, resulting in inefficiency and repeated human involvement. Challenge 3 is the scalability across programs. Most methods are tailored to specific datasets, lacking scalability across protocol programs, incurring adaptation costs that reduce the effectiveness of fuzzing in practical scenarios.

In this paper, we propose StatePre, an LLM-based state handling-method addressing precision, automation, and scalability limitations, which leverages the code comprehension capabilities of LLMs to automatically analyze and refine the state annotations in network protocol programs. To address Challenge 1, StatePre adopts a multi-stage strategy. For protocols with implicit states, it utilizes semantic role labeling powered by LLMs to infer states from textual descriptions, whereas for explicit-state protocols, it extracts state information by analyzing tables and diagrams in RFCs. Subsequently, it accurately maps these states to programs through a combination of static analysis and LLM-guided techniques, while ensuring all state transitions adhere to RFC specifications. To tackle Challenge 2, StatePre utilizes context-aware prompt engineering to automatically produce code-adaptive modification patches, removing the need for manual adjustments. Furthermore, it overcomes Challenge 3 through a structured knowledge alignment framework that enables adaptation across different protocol programs. By automatically refining state granularity and complementing missing state annotations through targeted code patching, StatePre facilitates accurate state-space exploration, thereby improving both the efficiency and effectiveness of fuzzing.

The evaluation results demonstrate that the programs modified with StatePre achieve an average state expansion of 170.18% and state transition enhancement of 128.30% compared to unmodified fuzzing programs. When evaluated on the ProFuzzBench dataset, StatePre-modified programs achieve 72.86% higher code coverage and detect 102.43% more unique crashes than those fuzzed without StatePre preprocessing. Furthermore, StatePre exhibits strong scalability, improving code coverage by 57.46% and crash discovery by 121.67% across 23 diverse network protocol programs.

The main contributions of this paper can be summarized as follows:

- We propose a novel LLM-based method to refine state granularity and complement missing state annotations, enabling precise state tracking during fuzzing.
- We implement a fully automated pipeline from code analysis to patch generation and execution, eliminating manual intervention and reducing preprocessing time from hours to minutes.
- We evaluate StatePre on 23 protocol programs from 10 different protocols, demonstrating good scalability and significant improvements in fuzzing effectiveness. StatePre also discovered previously unknown vulnerabilities in real-world programs.

2. Related Work and Motivation

2.1. Protocol Fuzzing

Fuzzing, a primary method for detecting vulnerabilities in network protocol programs, is widely recognized for its simplicity, efficiency and accuracy [6]. There are two dominant approaches—black-box fuzzing and grey-box fuzzing—which are distinguished by their differing levels of visibility of the internal workings of the target system [9]. Early network protocol fuzzers primarily operated as black-box tools, relying on randomized input generation and monitoring system responses without internal program analysis [10], such as SPIKE [11] and PROTOS [12]. However, these methods constrain path exploration and vulnerability detection due to the inability to inspect the program's internals [13]. To overcome these shortcomings, recent advancements have favored grey-box fuzzing, which incorporates partial internal program insights and runtime feedback [14]. A notable example, AFLNet [7], establishes foundational state-guided principles as an intelligent client that iteratively mutates message sequences while monitoring code coverage and state-space expansion through server responses, introducing state-aware feedback and coverage-guided fuzzing for stateful protocols, substantially outperforming traditional black-box approaches in branch coverage. AFLnet's derivative, StateAFL [8], advanced state modeling through runtime memory introspection, employing compile-time instrumentation to capture in-memory state representations and dynamically construct protocol state machines. Further innovation emerged through SGFuzz [15], which combines the static analysis of state variables with dynamic state transition table (STT) construction during fuzzing campaigns, enabling the systematic exploration of state-dependent code paths through hybrid static–dynamic analysis.

2.2. State Handling

Before fuzzing begins, state handling is a key factor that affects the effectiveness of network protocol fuzzing. State handling refers to the process of refining the state annotations in target programs by aligning them with the state definitions and descriptions specified in the RFCs. Existing approaches exhibit critical technical bottlenecks that hinder their practical utility.

First, the semantic gaps between protocol specifications and implementations remain unresolved. While RFCs formally define protocol behaviors, developers often misinterpret specifications or introduce ad hoc optimizations during implementation. For example, AFLNet [7] models protocol states using predefined response codes from RFCs, but this approach fails for protocols lacking explicit state identifiers (e.g., DNS), where states are implicitly managed through variable combinations or control flow patterns. Such inconsistencies distort state-space representations, causing coverage-guided fuzzers to miss critical execution paths.

Second, the limited automation in annotation refinement imposes efficiency barriers. ProfuzzBench [16], despite its precision, requires security experts to manually annotate

state variables—a process consuming over 40 person-hours per protocol according to empirical studies. This manual effort becomes unsustainable for evolving codebases, as even minor protocol updates necessitate re-annotation. ChatAFL [17] partially addresses this by leveraging LLMs to infer state transitions; yet, its reliance on predefined message sequences results in a 63% false-positive rate for protocols with implicit states like WebSocket [18], where state transitions depend on frame opcodes rather than explicit messages. This limited automation leads to a process of subsequent corrections that reduces the overall efficiency.

Third, the generalization capabilities are severely constrained by protocol-specific designs. Most tools exhibit narrow applicability. For instance, AFLNet excels with HTTP/SMTP but struggles with SIP's prose-based state definitions, while SGFuzz [15] requires protocol-specific static analysis templates. This forces practitioners to maintain multiple specialized tools, increasing operational complexity. The existing state-handling solutions cannot adapt to more than two protocol types without significant re-engineering.

The preceding analysis revealed three key challenges: imprecise state alignment (Challenge 1), non-automated annotation refinement (Challenge 2), and limited scalability across different programs (Challenge 3). These bottlenecks collectively undermine fuzzing efficacy, necessitating a unified solution that bridges specification–implementation gaps while ensuring automation and protocol-agnosticism.

2.3. Motivation

The current methods face significant challenges in state handling, which hamper their effectiveness.

Challenge 1 lies in achieving precise alignment between protocol specifications and program states. While RFC documents formally define protocol behaviors, developers often implement these specifications with varying interpretations, leading to mismatches in state granularity and missing critical transitions. For example, consider the following excerpt from RFC 7231 (Hypertext Transfer Protocol): "The 200 (OK) status code indicates that the request has succeeded. The payload sent in a 200 response depends on the request method. For the methods defined by this specification, the intended meaning of the 200 status code is defined in Section 6.3.1." In practice, developers might implement a single "200 OK" state to represent multiple successful operations, as shown in Listing 1.

Listing 1. Example of imprecise alignment between protocol specifications and program states.

```
if (operation_success) {
 set_state(200); // Generic success state
}
```

However, according to RFC 7231, more specific substates should be used to distinguish between different successful operations. For example, a resource creation should use the "201 Created" state, while a successful retrieval should use the "200 OK" state. The current approach of using a generic "200 OK" state for all successful operations leads to coarse-grained state annotations, which can obscure critical protocol semantics and limit the fuzzer's ability to generate context-aware test cases.

Challenge 2, non-automated annotation refinement, represents the second major hurdle. The existing state handling techniques rely heavily on manual effort, requiring security experts to painstakingly annotate state variables and transitions. This process not only consumes significant time and resources but also becomes unsustainable as protocol implementations evolve. The manual nature of current methods makes them impractical for modern development pipelines where continuous security testing is essential.

Challenge 3, limited scalability across different programs, severely restricts real-world applicability. Most state-handling solutions have been designed for specific protocols with

explicit state definitions (e.g., HTTP status codes), failing to generalize to protocols with implicit or complex state machines (e.g., DNS, WebSocket). This limitation forces security teams to maintain multiple specialized tools, increasing operational complexity and leaving many protocols inadequately tested.

To address the challenges mentioned above, we presents StatePre, an LLM-based state-handling method. Our key insight is leveraging LLMs' unique capabilities in both natural language processing (for RFC comprehension) and code understanding (for program analysis) to automatically bridge the semantic gap between specifications and protocol programs in state handling. StatePre enables conventional fuzzers to operate on properly enhanced program representations while maintaining their core exploration strategies.

3. Methodology

3.1. Overview Design

Figure 1 illustrates the workflow of StatePre, an LLM-based state-handling method for network protocol fuzzing. The framework consists of three main components: knowledge extraction, prompt and query construction, and patch execution. The workflow operates as follows:

Knowledge extraction. StatePre begins with knowledge extraction from protocol specifications, specifically RFC documents. This step is crucial for establishing a precise mapping between the protocol states defined in the RFCs and the corresponding code variable annotations in the target programs. By analyzing the structured and unstructured content of RFCs, StatePre identifies explicit state definitions and infers implicit states using semantic role labeling and other natural language processing techniques. This ensures that the state annotations in the code accurately reflect the protocol specifications, addressing the challenge of state alignment precision.

Prompt and Query Construction. After knowledge extraction, StatePre constructs prompts and queries tailored to address the limitations of the target programs. These limitations often include coarse-grained and missing state annotations, which hinder the effectiveness of fuzzing. StatePre generates context-aware prompts and queries that guide the LLM to produce modification descriptions. These descriptions aim to reconcile the program logic with the protocol specifications, ensuring that the state annotations are both RFC-compliant and semantically accurate. This step is essential for automating the refinement of state annotations and reducing manual intervention.

Patch execution. With the prompts and queries in place, StatePre proceeds to generate concrete patches. These patches are designed to refine existing state annotations and complement missing ones, particularly after critical state transitions. The patches are generated based on the modification descriptions provided by the LLM and are executed to update the target programs. This step ensures that the programs are instrumented with fine-grained state annotations, enabling the more precise tracking of protocol logic flows. The patched programs are then ready to enter the fuzzing loop.

Finally, the patched programs are integrated into the fuzzing loop of benchmark fuzzers. The fuzzing loop is fully automated and iterates until the desired state coverage is achieved or security-critical failures are identified. This integration ensures that the fuzzers can leverage the enhanced state annotations to explore the state space more effectively, leading to improved code coverage, accelerated crash discovery, and the better detection of state-dependent vulnerabilities.

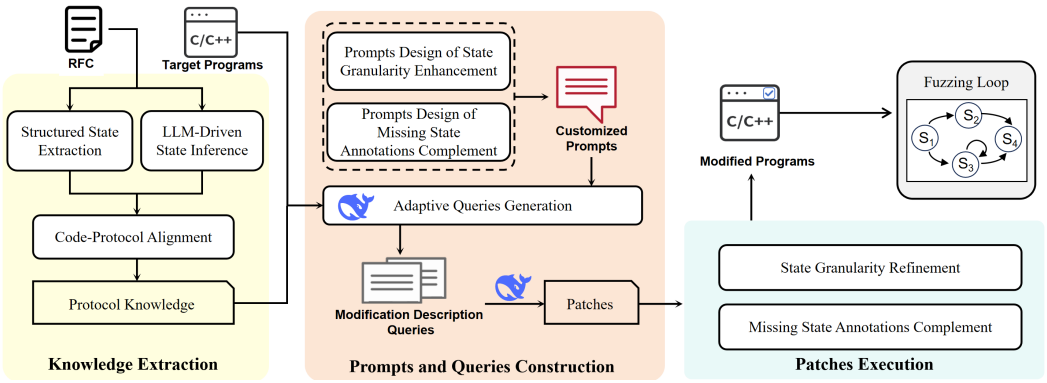


Figure 1. Workflow of StatePre.

3.2. Knowledge Extraction

The knowledge extraction phase systematically constructs a mapping between protocol specifications and program states through two complementary strategies, leveraging the analytical capabilities of LLMs to handle both structured and unstructured RFC content.

3.2.1. Structured State Extraction from RFCs

For RFCs with explicit state definitions, StatePre employs automated parsing to extract predefined state lists. The system processes RFC documents through two key structured extraction methods, tabular data extraction and state machine interpretation, to ensure precise alignment with protocol specifications. For tabular data extraction, it employs layout-aware parsing techniques to analyze the formal state tables embedded in RFC sections. This approach accurately converts HTML/PDF tables into structured JSON format while preserving the critical relationships between state codes, their symbolic names, and semantic descriptions.

For state machine interpretation, the solution implements vector graphics analysis to parse the technical diagrams found in RFC appendices. It identifies graphical components through geometric pattern recognition, converting states (represented as nodes) and transitions (shown as edges) into machine-readable adjacency lists. Each transition is annotated with its corresponding trigger messages or condition statements extracted from diagram labels. This dual approach, combining semantic table parsing with diagrammatic state machine reconstruction, guarantees the comprehensive coverage of all explicitly defined states and transitions in well-structured RFCs, maintaining strict adherence to protocol standards through systematic data normalization.

3.2.2. LLM-Driven State Inference

To enable LLMs to accurately infer protocol states from RFC documents, StatePre preprocesses raw RFC text into structured semantic chunks tailored for model comprehension. Each RFC section is segmented into functional units (e.g., authentication workflows, error handling clauses) and augmented with metadata, including protocol version, critical verbs (e.g., “MUST”, “SHALL”), and cross-referenced diagrams. For textual descriptions lacking explicit state definitions, the system constructs context-enriched prompts by combining RFC excerpts with code-variable mappings and protocol-specific schemas. As shown in Listing 2, the input to the LLM includes: the RFC excerpt (a verbatim snippet) and a JSON template enforcing RFC-compliant output formats, which is formatted as Listing 2.

Listing 2. Input example of RFC excerpt.

```
RFC excerpt: After receiving a USER command, the~server SHALL await a
     PASS command to verify credentials. Only authenticated clients
    MAY issue RETR or STOR~commands.

Generate state definitions adhering to RFC semantics using the
    following fields:
- State: unique identifier (red);
- Description: protocol behavior (blue);
- Transition triggers: commands/events;
- Next states: valid subsequent states;
- Allowed commands: post-transition permissions .
```

This structured input guides the LLM to produce outputs as Listing 3, where states (`AWAIT_AUTH`, `AUTHENTICATED`) and transitions are rigorously aligned with RFC requirements.

Listing 3. Example of LLM output for RFC excerpt.

```
State: AWAIT_AUTH
Description: Server waits for client authentication
Transition triggers: [USER, PASS]
Next states: [AUTHENTICATED]

State: AUTHENTICATED
Description: Client credentials verified
Allowed commands: [RETR, STOR]
```

Finally, cross-validation ensures robustness through consistency checks that flag conflicting transitions and program correlation that aligns inferred states with actual code variables using naming similarity metrics and control-flow pattern matching. The end-to-end process transforms ambiguous protocol descriptions into verifiable state machines while maintaining strict adherence to RFC semantics. Meanwhile, cross-validation through consistency checks and program correlation further mitigates the potential hallucinations of LLMs, ensuring inferred states strictly adhere to RFC semantics.

3.2.3. Code-Protocol Alignment

The code-protocol alignment phase establishes systematic mappings between the extracted/inferred protocol states and their corresponding program variables as annotations. For variables with RFC-compliant naming conventions, exact match linkages are directly created, such as binding a code variable named 201 to the RFC-defined “created” state with the description of "the request has been fulfilled and has resulted in one or more new resources being created. The primary resource created by the request is identified by either a location header field in the response or, if no location field is received, by the effective request URI". When naming diverges between specifications and programs, semantic bridging resolves mismatches using LLM-generated rationales that establish conceptual equivalence.

This process constructs a protocol knowledge base that rigorously aligns specification-defined states with their real-world code representations. By accommodating naming variations while preserving semantic fidelity, the system enables precise instrumentation across diverse programs, ensuring audit and testing frameworks operate against ground-truth protocol models.

3.3. Prompt and Query Construction

This phase designs targeted prompts and queries to guide the LLM in two critical tasks: state granularity refinement and missing state annotation complementing, addressing

scenarios where protocol programs either use coarse-grained state annotations or lack the necessary state annotations. The prompts and queries dynamically adapt to RFC content and programs, ensuring the precise alignment between specification requirements and code.

3.3.1. Prompt Design for State Granularity Enhancement

This phase designs prompts for the critical limitation of overgeneralized state annotations in protocol programs, which severely restricts the precision of state-aware fuzzing. Many programs conflate semantically distinct protocol states under a single state code—such as using HTTP 200 "OK" for all successful operations—despite RFC specifications defining finer-grained substates. This practice obscures nuanced states, state transitions, and response semantics, directly impacting the fuzzers' ability to explore the state space. In order for LLMs to better understand and master various fine-grained states in RFCs, it is necessary to design targeted prompts first. The design of prompts incorporates three modular components to enhance the precision and robustness:

Specification contextualization template. This template embeds protocol knowledge by injecting RFC excerpts and code context into the prompt's preamble. As shown in Listing 4, the template's triple-task structure (mapping, substitution, specification) ensures systematic analysis.

Listing 4. Specification contextualization template.

```
As a protocol security expert, analyze the following components:
 [RFC Excerpt] + {relevant_rfc_sections};
 [Code Snippet] + {target_code}.
Identify coarse-grained state codes.
Map code functionality to RFC substate semantics.
Propose compliant substates and specify required protocol elements.
```

Semantic bridging prompts. As shown in Listing 5, this prompt establishes traceability between program logic and protocol states through structured output formatting, enabling automated consistency checks.

Listing 5. Semantic bridging prompts.

```
Given {RFC_state_transition_diagram} and {code_control_flow}.
Generate mapping rules between code branches and RFC substates:
 Code condition: [conditional expression];
 RFC state: [substate identifier];
 Validation constraints: [RFC-defined invariants].
```

Validation-aware generation. As shown in Listing 6, this template ensures the generated patches comply with protocol state machines through explicit precondition/postcondition specifications.

Listing 6. Validation-aware generation template.

```
When refining {original_code} to use {target_substate}:
 List RFC-mandated headers/fields for {target_substate}.
 Identify code modifications and generate guard clauses enforcing:
 {required_pre_states} + {allowed_post_states}.
Output format:
 RFC requirements;
 Itemized list from RFC {section_number};
Code changes: Patch blocks with {target_substate} instrumentation.
State transition constraints:
 [Preconditions_boolean_expressions] + [Postconditions_state_assertions].
```

The system first identifies instances where programs use generic state codes to represent multiple RFC-defined substates. By cross-referencing the protocol knowledge base (Section 3.2) with code-level state assignments, it detects mismatches such as HTTP 200 being used in scenarios requiring specific success codes. The proposed prompts design employs a multi-stage decomposition approach, which separates state identification, mapping, and validation into discrete prompt phases. This modularization allows for the more systematic and focused handling of each aspect, thereby improving the overall accuracy and clarity of the prompts. Meanwhile, the prompt design leverages schema-guided generation to enforce structured outputs. By utilizing markdown formatting, it ensures that the generated prompts adhere to a consistent structure, facilitating automated parsing and reducing ambiguity. Furthermore, negative example injection is utilized to augment prompts with common program pitfalls, such as missing location headers in HTTP 201 responses. This technique enhances the robustness of the analysis by explicitly addressing potential errors and encouraging a more comprehensive evaluation.

3.3.2. Prompt Design of Missing State Annotation Complement

This phase focuses on designing prompts to address the absence of explicit state annotations in protocol programs, where critical state transitions are implicitly managed through coarse-grained variables or control flow rather than standardized annotations. The proposed prompt architecture consists of three modular templates that systematically bridge program-specific logic with RFC-defined states:

Implicit state identification template. As shown in Listing 7, this template detects latent state transitions by correlating variable operations and API call patterns with protocol specifications to identify variables such as `auth_flag` and API calls like `start_data_session()` as implicit state handlers, anchoring them to the authentication states defined in RFCs via cross-referencing between code and the RFC standard.

Listing 7. Implicit state identification template.

```
As a protocol conformance analyst, analyze [RFC excerpts] + [Code].
Identify implicit state indicators:
 Variables controlling protocol phases;
 Critical API calls altering operational modes.
Map each indicator to RFC states {target_states} based on:
 Preconditions in RFC Section {X.Y};
 Post-state behaviors in RFC Section {Z.W}.
Propose implicit state indicators for state logging with format
 Variable: {var_name} modified at line {N};
 API Call: {function_name} invoked at line {M}.
RFC Compliance Mapping: {code_element} -> {rfc_state} (RFC {section})
```

Semantic anchoring prompts. As shown in Listing 8, this phase establishes formal mappings between program patterns and protocol states to create explicit links between variable states and RFC states, enforcing valid transition sequences through precondition checks.

Constraint-aware injection template. As shown in listing 9, this template injects RFC-compliant annotations and validation logic to transform implicit transitions into auditable state markers.

Listing 8. Semantic anchoring prompts.

```
Given variable lifecycle analysis:
 Declaration: {var_declaration};
 Write sites: {write_points};
 Read contexts: {read_contexts};
 RFC state machine: {state_machine}.
Generate binding rules adhering to: <Mapping Rule Format>
 Code pattern: {code_pattern};
 RFC state: {rfc_state}.
Transition constraints: {required_predecessors}; {allowed_successors}
```

Listing 9. Constraint-aware injection template.

```
Augment the following code with state annotations:{original_code}
Required modifications:
 Insert LOG_STATE({target_state}) at line {L}
 Add precondition validation before {critical_function}:
 if CURRENT_STATE != {required_state}: {abort_function}()
 Enforce post-state assertions after {transition_point}

Output format:
 Line {X}: LOG_STATE({state}) // Derived from RFC
 Line {Y}: ENFORCE_STATE({state})

Transition contracts: {Precondition}; {Postcondition}
```

This design methodology is guided by three core principles: contextual anchoring, temporal consistency, and machine-actionable outputs. Contextual anchoring involves the joint analysis of variable lifecycles and API call chains to surface implicit states. Temporal consistency is achieved by encoding RFC state sequences as preconditions and postconditions in the generated code. Finally, machine-actionable outputs are enabled through structured markdown formatting, which facilitates the automated instrumentation of annotations.

3.3.3. Adaptive Query Generation

The system dynamically tailors queries based on the structural characteristics of RFC specifications and program patterns, enabling protocol-agnostic state instrumentation. For protocols with explicit RFC state lists such as HTTP, StatePre directly maps code variables to predefined state codes through automated table alignment. This allows the generation of surgical instrumentation queries targeting known transition points. The queries focus exclusively on annotation placement at these verified state transition boundaries, leveraging the protocols' standardized state definitions. For protocols with descriptive RFC specifications like SIP, where states are defined through prose rather than explicit codes, the system first constructs an intermediate state schema via LLM's summarization of RFCs. Upon encountering requirements, the LLM generates a state machine skeleton that guides the construction of targeted instrumentation directives linked to program logic. The adaptive strategy ensures consistent annotation quality across both structured and unstructured protocol specifications, eliminating manual template engineering while maintaining strict RFC compliance.

3.4. Patches Execution

This phase, utilizing concrete patches transformed from the LLM-generated output of modification descriptions, focuses on two primary tasks: state granularity refinement and missing state annotation complement, ensuring the instrumented code accurately reflects protocol specifications while maintaining functional equivalence.

3.4.1. State Granularity Refinement

This phase targets overgeneralized state annotations in protocol programs that undermine fuzzing precision by conflating semantically distinct protocol states. A prevalent example is the misuse of HTTP 200 "OK" as a catch-all success code, despite RFC 7231 defining nuanced substates for specific operational contexts. Such coarse-grained state representations obscure critical protocol semantics, limiting the fuzzer's ability to generate context-aware test cases. StatePre addresses this by refining programs to adopt RFC-compliant substates through a systematic detection, analysis, and transformation pipeline.

The process begins with coarse-grained state detection, where the system cross-references code-level state assignments with the protocol knowledge base (Section 3.2). For HTTP programs, this identifies instances where developers incorrectly use state code 200 for operations requiring specific success indicators. StatePre flags this mismatch through RFC mapping rules specifying that resource creation demands state 201 "Created". Subsequent semantic grounding analyzes the code context to ascertain the eligibility for refinement. The LLM examines function names, API calls, and data operations to verify compliance with the RFC 201 semantics. This contextual verification helps avoid false positives, differentiating, for instance, between actual resource creation that should use 201 and simple data updates that might legitimately use 200. Once confirmed, precise code modifications are executed to achieve three critical changes: replacing the generic code with its RFC-specified counterpart, injecting protocol elements as mandated by RFC requirements, and inserting instrumentation for state-specific logging annotations. The transformation yield is shown in Listing 10.

Listing 10. Example of state granularity refinement for programs based on RFC with descriptive specifications.

```
if (save_resource(req)) {
 set_state(201); // RFC 7231-compliant code
 add_header("Location", new_resource); // Required for 201 responses
 LOG_STATE(HTTP_201); // Granular state tracking
}
```

Validation ensures the modified states adhere to the RFC transition rules through automated consistency checks. For HTTP, this verifies that 201 responses include the mandatory location header and that the subsequent state transitions align with protocol workflows. By differentiating between substates like 202 "Accepted" (asynchronous processing) and 206 "Partial Content" (range requests), this refinement enables the fuzzer to apply state-specific mutation strategies. For instance, 206 responses trigger specialized boundary checks for content-range headers, while 202 scenarios test asynchronous job polling mechanisms. Even if LLM-generated substates contain rare inaccuracies, the instrumentation preserves functional equivalence and merely introduces auxiliary logging, leaving crash analysis unaffected while potentially diversifying fuzzing exploration.

3.4.2. Missing State Annotation Complement

The patch engine injects state-tracking logic through context-aware code modifications, ensuring the precise capture of protocol state transitions while preserving program functionality. For explicit state variable assignments, the system appends logging statements directly after state changes to record RFC-compliant identifiers. For implicit state transitions triggered by function calls or indirect modifications, the engine generates wrapper functions to intercept state changes.

The instrumentation strategically places logging statements after security-critical operations but before error returns to avoid polluting failure paths. The three-tiered

approach—direct assignment instrumentation, function wrapping, and control flow-aware placement—guarantees comprehensive state visibility while maintaining runtime efficiency and code stability.

4. Evaluation

4.1. Implementation

The StatePre framework was implemented as a modular pipeline integrating LLMs with protocol-aware code analysis tools. The core system comprised 4800 lines of C++ code and 500 lines of Python 3.11.8 scripts, built atop the LLVM framework [19], to enable precise code instrumentation and static analysis. Key components included a state-sensitive code locator for identifying state-related variables, an LLM-guided code modifier for generating RFC-compliant patches, and an automated validation engine for ensuring functional equivalence.

For LLM integration, StatePre supports multiple state-of-the-art models, including DeepSeek-R1 [20], Gemma [21], Llama 3 [22], Qwen 2.5 [23], Claude [24], and GPT-3.5 [25], each configured with a temperature of 0.2 and top-p sampling of 0.9 to balance creativity and consistency. Empirical tests across these models demonstrated comparable performance in generating valid state annotations, with less than 5% variance in patch accuracy, underscoring the robustness of our prompt engineering strategy. For instance, when refining HTTP state codes, all models achieved over 92% compliance with RFC 7231 semantics, validating their interchangeability in the pipeline.

The framework interfaces with benchmark fuzzers AFLnwe and AFLnet through a shared instrumentation layer. During preprocessing, StatePre injects lightweight logging hooks into target programs to track state transitions, adding less than 3% runtime overhead. Critical parameters, such as the maximum iteration count for LLM-based refinement (set to 10) and the state coverage threshold (95%), are user-configurable to adapt to diverse protocol requirements.

4.2. Experiment Setup

To comprehensively evaluate the effectiveness of StatePre, we conducted a series of experiments designed to answer the following research questions:

- **RQ1, effectiveness of state-related code localization and modification in StatePre:** could StatePre effectively locate and modify state-related code based on its code analysis and transformation techniques?
- **RQ2, granularity of state space exploration in StatePre:** could StatePre achieve finer-grained state space exploration by leveraging its state-space exploration strategy?
- **RQ3, fuzzing performance enhancement with StatePre:** could StatePre enhance overall fuzzing performance through its state-aware fuzzing optimization framework?
- **RQ4, protocol-agnostic scalability of StatePre:** could StatePre maintain scalability across diverse protocol programs based on its protocol-agnostic state model adaptation approach?

4.2.1. Dataset

To address these questions, we selected a diverse dataset comprising 23 network protocol programs from 10 different protocols, including FTP, SMTP, SIP, and others. These programs were selected from the ProFuzzBench benchmark, which is widely recognized for its comprehensive coverage of stateful protocols. To ensure a thorough evaluation, we expanded the original 13 programs from ProFuzzBench to include 10 additional programs, covering a broader range of protocols and codebases. Table 1 shows the information of the target programs.

Table 1. Basic information on target network protocol programs for evaluation.

Protocol Type	Program	Version/Commit
DAAP	forked-daapd	2ca10d9
DICOM	dcmqrscp	7f8564c
DNS	dnsmasq	v2.73rc6
DTLS	TinyDTLS	06995d4
FTP	ProFTPD	4017eff8
FTP	LightFTP	5980ea1
FTP	Pure-FTPD	c21b45f
FTP	BFTPD	v5.7
RTSP	Live555	ceeb4f4
SIP	Kamailio	2648eb3
SMTP	Exim	38903fb
SSH	OpenSSH	7cfea58
TLS	OpenSSL	437435
DAAP	Firefly Media Server	v4.9.6
DICOM	Conquest DlCOM Server	e98sf4w8
DNS	BIND	1ds6f1es
DTLS	GnuTLS	62395874
FTP	FileZilla	kanj322
RTSP	MistServer	v5.4.1-e-733b
SIP	ChimeraSIP	3.5.7-klmno
SMTP	OpenSMTPD	2c3d4e5
SSH	Dropbear	6f7a8b9c1
TLS	BoringSSL	0f1e2d3a9

4.2.2. Comparison Methods

For comparison, we established four groups of protocol programs: the first group consists of the original unmodified programs, serving as a baseline, which are also usually directly used in protocol fuzzing; the second group includes programs modified with manual state patches from ProFuzzBench, representing the state-of-the-art manual approach; the third group comprises programs modified by our automated instrumentation pipeline, StatePre-Auto; and the fourth group includes programs enhanced through manual refinement by experts, denoted as Expert-Refined. This setup allows us to compare StatePre against both manual and automated state handling methods, providing a comprehensive assessment of its performance.

4.2.3. Evaluation Metrics

The evaluation metrics used in our experiments include code coverage, unique crash discovery, and state space exploration. Code coverage measures the proportion of code executed during fuzzing, indicating how thoroughly the program's logic has been tested. Unique crash discovery counts the number of distinct crashes found, reflecting the effectiveness of vulnerability detection. State space exploration assesses the granularity and diversity of states explored during fuzzing, highlighting the precision of state handling. These metrics collectively provide a holistic view of StatePre's impact on fuzzing efficiency and effectiveness.

4.2.4. Experimental Configuration

The experiments were conducted on a high-performance testing machine equipped with 128 Intel(R) Xeon(R) Platinum 8358 CPUs and 384 GB of memory, ensuring sufficient computational resources for the evaluation. The operating system used was Linux Ubuntu 20.04 $\times$ 86/64. Each target program was fuzzed with the benchmark fuzzers AFLnwe and AFLnet for 24 h, and the process was repeated four times to ensure statistical reliability. This

rigorous setup ensured that our results were robust and reflective of real-world scenarios. To provide a clear summary of the experimental setup, we present the hardware and docker configurations in Table 2.

Table 2. Experimental hardware and docker configurations.

Component	Specification
CPU	128 Intel(R) Xeon(R) Platinum 8358 CPUs
Memory	384 GB of RAM
Storage	SSD Disk
Operating System	Linux Ubuntu 20.04 X86/64
Resource Allocation	Same computational resources for each Docker container
Fuzzing Duration	24 h per target program
Repetitions	4 times per target program

By carefully selecting the dataset, comparison methods, evaluation metrics, and experimental environment, we aimed to conduct a comprehensive and objective assessment of StatePre's contributions to network protocol fuzzing.

4.3. Evaluation on Locating and Modifying State-Related Code ***(RQ1)***

To validate the effectiveness of StatePre in automating state-handling code modifications, we conducted a systematic evaluation across two datasets: the original 13 programs from ProFuzzBench and an extended set of 23 protocol programs. The results demonstrate StatePre's capability to accurately identify and modify state-related code while significantly outperforming manual approaches.

4.3.1. Static Analysis and Modification Coverage

Table 3 summarizes StatePre's performance in detecting and modifying state-handling issues. For the 13 ProFuzzBench targets, we identified a total of 342 state-related code segments through static analysis. StatePre detected 326 of these state-related code segments and automatically generated patches for them, achieving a modification completion rate of 95.32%. In contrast, ProFuzzBench's manual modifications addressed only seven segments (2.05% coverage). When extended to 23 programs (including complex codebases like BIND and FileZilla), StatePre maintained a 94.89% coverage (576/607 issues), proving its scalability.

Table 3. State-related code identification and modification results.

Dataset	Targets	Total State Issues	ProFuzzBench Modifications	Coverage (ProFuzzBench)	StatePre Modifications	Coverage (StatePre)
ProFuzzBench	13	342	7	2.05%	326	95.32%
Extended Dataset	23	607	-	-	576	94.89%

ProFuzzBench's manual approach lacked scalability, requiring domain expertise and extensive time to cover even 2% of the issues. In contrast, StatePre's integration of protocol specifications and LLM-based code understanding enabled the precise identification of state variables, transitions, and dependencies. For example, in OpenSSL, it detected 28 state handlers that were overlooked in the manual annotations. However, the 4.68% unmodified issues primarily involved implicit state dependencies or non-standard protocol extensions.

4.3.2. Code Modification Accuracy and Validation

StatePre's generated patches were rigorously validated through code reviews and unit testing. All modifications passed syntactic and semantic checks with 100% accuracy, confirming no introduction of logic errors. Figure 2 illustrates the relationship between actual state issues, ProFuzzBench's manual fixes, and StatePre's automated modifications. StatePre not only fully subsumed ProFuzzBench's limited patches but also addressed 94.98% of the actual issues that needed to be resolved. All modifications passed crash reproducibility tests, confirming that any residual annotation errors (e.g., misclassified substates) did not alter crash root causes or impede vulnerability triage.

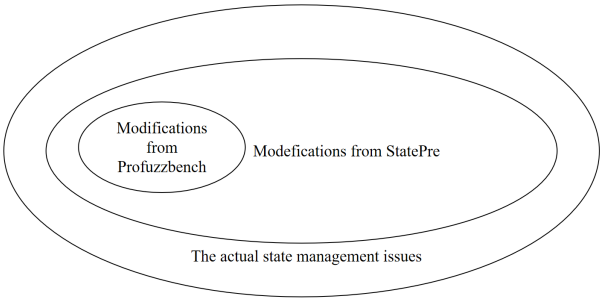


Figure 2. Coverage of actual state issues by ProFuzzBench and StatePre.

4.3.3. Efficiency and Practical Impact

StatePre reduced manual effort by 92.60% compared to ProFuzzBench, requiring only 3–8 min per target for validation (vs. hours for manual analysis). The framework processed large codebases efficiently, e.g., OpenSSL's 600k+ LoC in 5.8 min, demonstrating practical applicability and efficiency.

This evaluation confirms that StatePre effectively automates the identification and modification of state-related code with high precision and scalability. Its LLM-driven approach translates protocol semantics into actionable code changes, addressing the limitations of manual methods and laying a foundation for enhanced fuzzing efficacy.

4.4. Evaluation of State Annotation Granularity ***(RQ2)***

To assess the impact of StatePre on refining state-handling granularity, we analyzed the state-space exploration capabilities of the protocol programs before and after preprocessing. The results demonstrate significant improvements in both state diversity and transition coverage during fuzzing.

4.4.1. State Granularity Improvement

Table 4 compares the number of states and state transitions observed during fuzzing for ProFuzzBench's manually modified programs versus StatePre-preprocessed programs. StatePre achieved an average state expansion rate of 170.18% and a state transition increase rate of 128.30%, enabling the finer-grained monitoring of protocol behaviors.

Table 4. State Granularity Improvement.

Program	States (ProFuzzBench)	States (StatePre)	State Increase Rate	State Transitions (ProFuzzBench)	State Transitions (StatePre)	State Transition Increase Rate
forked-daapd	9	19	111.11%	38	88	131.58%
dcmqrscp	8	20	150.00%	34	73	114.71%
dnsmasq	11	22	100.00%	45	106	135.56%
TinyDTLS	7	18	157.14%	28	62	121.43%
ProFTPD	10	25	150%	43	99	130.23%
LightFTP	11	27	145.45%	45	100	122.22%
PureFTPD	12	26	116.67%	51	113	121.57%
BFTPD	9	20	122.22%	38	86	126.32%
Live555	6	27	350.00%	29	76	162.07%
Kamailio	6	20	233.33%	24	51	112.50%
Exim	9	26	188.89%	37	87	135.14%
OpenSSH	8	25	212.50%	33	72	118.18%
OpenSSL	8	22	175.00%	33	78	136.36%
Average	-	-	170.18%	-	-	128.30%

StatePre increased the granularity of distinguishable states by 3.5× in Live555 (RTSP), splitting coarse states such as "CONNECTION_OPEN" into more detailed substates like "MEDIA_SESSION_INIT" and "RTP_STREAM_ACTIV". This enhanced state tracking also enabled deeper exploration of protocol phases, uncovering 45 new transitions related to post-handshake message processing in OpenSSL (TLS). The improvements in state granularity were most pronounced in stateful protocols, with RTSP seeing a 350% increase in states and SIP experiencing a 233.33% increase.

4.4.2. State-Space Exploration Analysis

Figure 3 compares the Input-to-State Property Mapping (IPSM) diagrams of the original Live555 program and its StatePre-preprocessed version. The IPSM of StatePre-preprocessed program exhibits 27 states (vs. 6 originally) and 76 transitions (vs. 29 originally), showing finer-grained distinctions and capturing more nuanced interactions.

By refining state granularity and complementing state annotations, under the same conditions, the programs processed by StatePre were able to be explored to a finer-grained and richer state space by the benchmark fuzzers. This was verified experimentally: by comparing the IPSM graph of the original program and the program processed by StatePre, we could see that the processed program increased not only the number of states but also the number of state transitions (edges). This comprehensive exploration of the state space provides a solid foundation for improving fuzzing efficiency and also brings hope for discovering potential security vulnerabilities. Therefore, StatePre shows potential to improve the test effect in network protocol fuzzing, paving the way for more effective fuzzing.

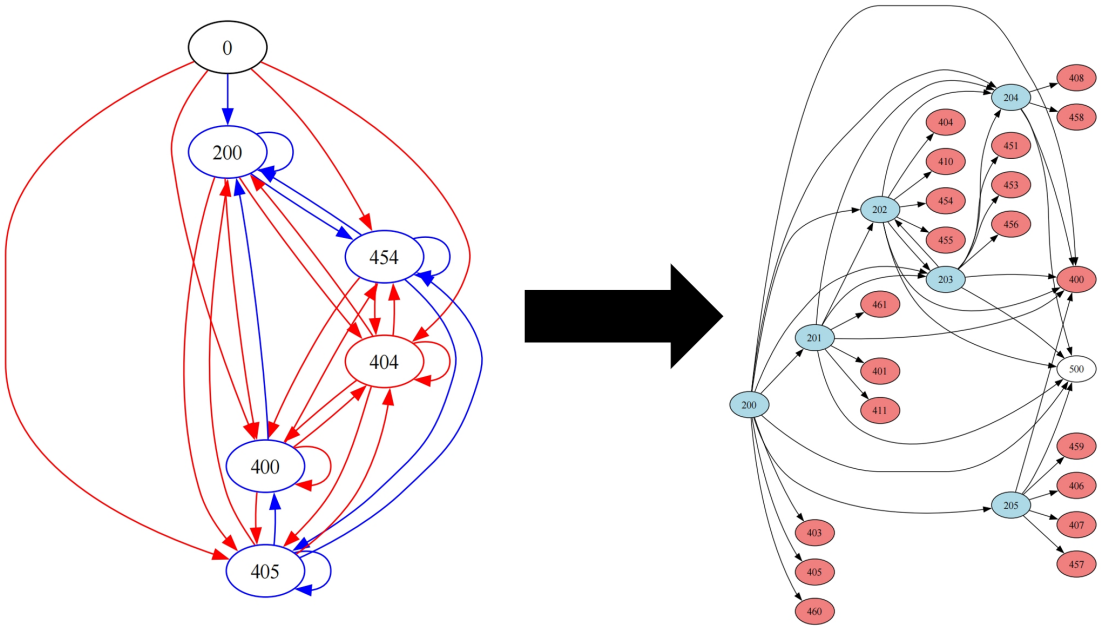


Figure 3. IPSM comparison for Live555 before and after StatePre preprocessing.

4.5. Evaluation on Enhancement of Fuzzing ***(RQ3)***

This experiment evaluated the practical impact of StatePre's preprocessing on network protocol fuzzing efficacy, including improvements in code coverage, unique crash discovery, and operational efficiency. We compared fuzzing outcomes under identical time and resource constraints before and after applying StatePre. The key results are summarized in Table 5.

Table 5. Fuzzing efficacy improvements.

Program	Code Coverage (ProFuzzBench)	Code Coverage (StatePre)	Code Coverage Improvement	Unique Crashes (ProFuzzBench)	Unique Crashes (StatePre)	Unique Crash Increase Rate
forked-daapd	7.57%	14.38%	89.96%	116	249	114.66%
dcmqrscp	7.08%	13.06%	84.46%	73	152	108.22%
dnsmasq	8.05%	14.76%	83.35%	77	146	89.61%
TinyDTLS	8.68%	14.92%	71.89%	72	143	98.61%
ProFTPD	9.52%	13.92%	46.22%	111	251	126.13%
LightFTP	7.87%	13.85%	75.98%	82	162	97.56%
PureFTPD	9.5%	15.49%	63.05%	73	149	104.11%
BFTPD	8.93%	15.56%	74.24%	102	184	80.39%
Live555	7.46%	15.74%	110.99%	112	215	91.96%
Kamailio	8.2%	14.37%	75.24%	65	137	110.77%
Exim	9.03%	14.06%	55.70%	107	210	96.26%
OpenSSH	9.33%	13.74%	47.27%	62	120	93.55%
OpenSSL	8.16%	13.78%	68.87%	91	200	119.78%
Average			72.86%			102.43%

StatePre demonstrated notable advancements in code coverage expansion, crash discovery acceleration, and operational efficiency. Across all targets, it achieved an average code coverage improvement of 72.86% and a 102.43% increase in unique crashes discovered. To evaluate the statistical significance of these improvements, we conducted a paired *t*-test comparing the results between the ProFuzzBench baseline and StatePre-preprocessed programs. The results showed that the improvements were statistically significant with a

p-value of less than 0.01, indicating that StatePre's enhancements were not due to random chance. Furthermore, StatePre significantly improved operational efficiency by reducing preprocessing time to under 10 min per target, compared to manual efforts that require dozens of hours. For instance, OpenSSL's codebase, which exceeds 600,000 lines of code, was processed in just 11.8 min, underscoring its scalability for large-scale deployments.

The results confirm that StatePre's automated preprocessing directly translates to practical security gains. By resolving state-handling limitations, it empowers fuzzers to uncover more crashes. The framework's efficiency gains also enable rapid iteration for emerging threats, reducing analyst workload while maximizing testing ROI.

4.6. Evaluation on Scalability ***(RQ4)***

This experiment evaluated StatePre's adaptability across diverse network protocol programs by measuring improvements in code coverage and unique crash discovery after preprocessing. Table 6 compares fuzzing outcomes before and after applying StatePre to an expanded dataset, including large-scale and niche protocol programs.

Table 6. Scalability assessment results.

Protocol	Program	Code Coverage Improvement	Crash Increase Rate
DAAP	Firefly Media Server	61.88%	116.44%
DICOM	Conquest DICOM Server	56.55%	114.63%
DNS	BIND	45.40%	108.60%
DTLS	GnuTLS	51.62%	108.54%
FTP	FileZilla	64.59%	169.57%
RTSP	MistServer	72.16%	120.45%
SIP	ChimeraSIP	58.01%	113.51%
SMTP	OpenSMTPD	53.38%	108.97%
SSH	Dropbear	50.50%	128.99%
TLS	BoringSSL	60.50%	127.03%
Average		57.46%	121.67%

StatePre showcased broad protocol support and the robust handling of complexity. Across 10 additional programs beyond the original ProFuzzBench dataset, it achieved an average code coverage improvement of 57.46% and a 121.67% increase in crash discovery. To further validate the scalability of StatePre, we conducted a paired *t*-test on the code coverage and unique crash discovery results for these additional programs. The results showed that the improvements in scalability were statistically significant with a *p*-value of less than 0.05, confirming StatePre's ability to generalize across diverse protocol programs. The framework also demonstrated consistent efficacy on large codebases like BIND (DNS) and niche protocols like Conquest DICOM Server, with coverage improvements exceeding 45% in all cases. This highlights StatePre's ability to generalize across diverse protocol semantics and code structures.

StatePre's scalability stems from its protocol-agnostic design, which automates state-dependency extraction without manual intervention. For instance, it correctly identified BIND's implicit state transitions tied to DNS query counters, enabling a 45.4% coverage boost. The framework's efficiency remained stable even for complex codebases like FileZilla (200k LoC), requiring under 15 min for preprocessing—orders of magnitude faster than manual approaches. This evaluation confirms StatePre's capability to enhance fuzzing efficacy across diverse network protocols, making it a versatile tool for large-scale security testing.

Furthermore, regarding the unique crashes discovered in the experiments, due to time constraints, we analyzed two programs' unique crashes and identified two new vulnerabilities: in ProFTPD version 1.3.5b and below, there exists an integer overflow in the MLSD module of the `mod_ls` module, enabling a remote attacker to cause a denial-of-service (crash); in LiveNetworks Live555 Streaming Media version 2013.11.25 and earlier, within the `lookupServerMediaSession` function, when fileExists is true and smsExists is also true, but `isFirstLookupInSession` is true, the function first calls `removeServerMediaSession(sms)` to remove the existing ServerMediaSession, then attempts to create a new `ServerMediaSession`. However, if the creation of the new `ServerMediaSession` fails (for instance, due to memory allocation failure), the function does not perform proper memory deallocation for the old `ServerMediaSession`, leading to a memory leak. Over time and with multiple calls to this function, the memory leak accumulates, potentially leading to program crashes. Due to time constraints, we could not analyze all crashes. We prepared detailed materials and reported these findings to the relevant departments as well as concurrently published all discovered crashes and related PoCs at https://gitee.com/haideonrubbish/StatePre (accessed on 29 April 2025).

5. Future Work

In certain network protocol fuzzing scenarios, it is challenging to ascertain the protocols of target programs in advance, and the evolution and updates of network protocols are also critical aspects that require attention. Accordingly, the StatePre framework suggests two promising directions for future research that are recommended for further exploration. First, automated protocol mechanism identification addresses scenarios where the target protocol is unknown. By integrating deep learning techniques [26], protocol reverse engineering can be significantly enhanced. For instance, leveraging neural networks [27] for protocol field recognition [28] and state machine inference [29] would reduce the dependency on prior protocol knowledge while improving analysis precision. Second, protocol-aware adaptive learning [30] focuses on developing self-evolving mechanisms to handle protocol variations and version updates. Techniques such as active learning [31] and reinforcement learning [32] could enable dynamic adaptation to emerging protocol semantics, creating an intelligent system capable of autonomously exploring new protocol logic without manual reconfiguration. These focused enhancements aim to strengthen StatePre's applicability in real-world environments while preserving its core methodological rigor. The proposed directions maintain technical coherence with the original framework, avoiding overextension into tangential research domains.

6. Conclusions

In this paper, we address the critical challenge of the imprecise state tracking in network protocol fuzzing by proposing StatePre, a novel state-handling method that leverages LLMs to automate state refinement through protocol specification analysis and code patching. Our approach combines RFC document parsing with semantic-aware state variable identification to generate fine-grained state annotations without manual intervention. Evaluated on 23 real-world protocol programs, StatePre enables conventional fuzzers to achieve deeper protocol logic exploration, higher code coverage, improved detection of crashes, and higher efficiency. Furthermore, this approach facilitated the discovery of crashes in production-grade protocol programs, proving its value for existing fuzzing frameworks.

Author Contributions: Conceptualization, Y.Z., K.Z. and J.P.; methodology, Y.Z., K.Z. and J.P.; software, Y.Z. and J.P.; validation, Y.Z., K.Z. and J.P.; formal analysis, Y.Z., J.P. and Q.C.; investigation, Y.Z., J.P. and Z.L.; resources, Y.Z. and Y.L.; data curation, Y.Z. and Z.L.; writing—original draft preparation, Y.Z.; writing—review and editing, K.Z., Y.L. and Q.C.; visualization, K.Z.; supervision, K.Z., Y.L. and Q.C.; project administration, K.Z. and Y.L. All authors have read and agreed to the published version of this manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: No new datasets were created in this study. All real-world applications can be searched and downloaded based on the version numbers, and the experimental data and code were published: https://gitee.com/haideonrubbish/StatePre (accessed on 29 April 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Qin, S.; Hu, F.; Ma, Z.; Zhao, B.; Yin, T.; Zhang, C. Nsfuzz: Towards efficient and state-aware network service fuzzing. *ACM Trans. Softw. Eng. Methodol.* **2023**, *32*, 1–26. [CrossRef]
2. Jiang, S.; Zhang, Y.; Li, J.; Yu, H.; Luo, L.; Sun, G. A Survey of Network Protocol Fuzzing: Model, Techniques and Directions. *arXiv* **2024**, arXiv:2402.17394. [CrossRef]
3. Faturrohman, M.; Salsabila, A.; Mardiah, Z.; Kardian, A.R. Attack in to The Server Message Block (CVE-2020-0796) Vulnerabilities in Windows 10 using Metasploit Framework. *JEEMECS (J. Electr. Eng. Mechatron. Comput. Sci.)* **2023**, *6*, 37–44. [CrossRef]
4. Tan, C.C.; Selvarajah, V. Wannacry ransomware attack: The enemy lies under your blanket. In *Proceedings of the AIP Conference*; AIP Publishing: Melville, NY, USA, 2024; Volume 2802.
5. Ma, Y.; Ma, R.; Lin, Z.; Zhang, R.; Cai, Y.; Wu, W.; Wang, J. Improving age of information for covert communication with time-modulated arrays. *IEEE Internet Things J.* **2024**, *12*, 1718–1731. [CrossRef]
6. Serebryany, K. libFuzzer: A Library for Coverage-Guided Fuzz Testing. In Proceedings of the 2016 USENIX Workshop on Offensive Technologies (WOOT), Austin, TX, USA, 8–9 August 2016; USENIX Association: Berkeley, CA, USA, 2016; pp. 1–10.
7. Pham, V.T.; Böhme, M.; Santosa, A.E.; Cîrstea, A.; Roychoudhury, A. AFLNet: A Greybox Fuzzer for Network Protocols. In Proceedings of the 2020 IEEE International Conference on Software Testing, Validation and Verification (ICST), Porto, Portugal, 24–28 October 2020; pp. 460–465. [CrossRef]
8. Natella, R. Stateafl: Greybox fuzzing for stateful network servers. *Empir. Softw. Eng.* **2022**, *27*, 191. [CrossRef]
9. Zhao, X.; Qu, H.; Xu, J.; Li, X.; Lv, W.; Wang, G.G. A systematic review of fuzzing. *Soft Comput.* **2024**, *28*, 5493–5522. [CrossRef]
10. Pereyda, J. Boofuzz: Network Protocol Fuzzing for Humans. 2023. Available online: https://github.com/jtpereyda/boofuzz (accessed on 30 April 2025).
11. Aitel, D. The advantages of block-based protocol analysis for security testing. *Immun. Inc. Febr.* **2002**, *105*, 106.
12. Kaksonen, R.; Laakso, M.; Takanen, A. Software security assessment through specification mutations and fault injection. In Proceedings of the Communications and Multimedia Security Issues of the New Century: IFIP TC6/TC11 Fifth Joint Working Conference on Communications and Multimedia Security (CMS'01), Darmstadt, Germany, 21–22 May 2001; Springer: Berlin/Heidelberg, Germany, 2001; pp. 173–183.
13. Liu, K.; Yang, M.; Ling, Z.; Zhang, Y.; Lei, C.; Luo, J.; Fu, X. Riotfuzzer: Companion app assisted remote fuzzing for detecting vulnerabilities in iot devices. In Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, Salt Lake City, UT, USA, 14–18 October 2024; pp. 2341–2354.
14. Banks, G.; Cova, M.; Felmetsger, V.; Almeroth, K.C.; Kemmerer, R.A.; Vigna, G. Snooze: Toward a Stateful Network Protocol Fuzzer. In Proceedings of the 9th International Conference on Information Security (ISC 2006), Seoul, Republic of Korea, 19–22 October 2006; Lecture Notes in Computer Science; Volume 4176, pp. 343–358. [CrossRef]
15. Ba, J.; Böhme, M.; Mirzamomen, Z.; Roychoudhury, A. Stateful greybox fuzzing. In Proceedings of the 31st USENIX Security Symposium (USENIX Security 22), Boston, MA, USA, 10–12 August 2022; pp. 3255–3272.
16. Natella, R.; Pham, V.T. Profuzzbench: A benchmark for stateful protocol fuzzing. In Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing And Analysis, Seattle, WA, USA, 11–17 July 2021; pp. 662–665.
17. Meng, R.; Mirchev, M.; Böhme, M.; Roychoudhury, A. Large language model guided protocol fuzzing. In Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS), San Diego, CA, USA, 26 February–1 March 2024; Volume 2024.

18. Fette, I.; Melnikov, A. *The Websocket Protocol*; Technical Report RFC 6455; Internet Engineering Task Force (IETF): Fremont, CA, USA, 2011. Available online: https://tools.ietf.org/html/rfc6455 (accessed on 5 May 2025).
19. Lattner, C.; Adve, V. LLVM: A compilation framework for lifelong program analysis & transformation. In Proceedings of the International Symposium on Code Generation and Optimization, 2004, San Jose, CA, USA, 20–24 March 2004; pp. 75–86.
20. Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv* **2025**, arXiv:2501.12948.
21. Team, G.; Mesnard, T.; Hardin, C.; Dadashi, R.; Bhupatiraju, S.; Pathak, S.; Sifre, L.; Rivière, M.; Kale, M.S.; Love, J.; et al. Gemma: Open models based on gemini research and technology. *arXiv* **2024**, arXiv:2403.08295.
22. Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. The llama 3 herd of models. *arXiv* **2024**, arXiv:2407.21783.
23. Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; et al. Qwen2. 5 technical report. *arXiv* **2024**, arXiv:2412.15115.
24. Anthropic. Claude 2. 2023. Available online: https://www.anthropic.com/claude (accessed on 1 September 2023).
25. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models are Few-Shot Learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901.
26. LeCun, Y.; Bengio, Y.; Hinton, G. Deep learning. *Nature* **2015**, *521*, 436–444. [CrossRef] [PubMed]
27. Zhang, G.P. Neural networks for classification: A survey. *IEEE Trans. Syst. Man. Cybern. Part C (Appl. Rev.)* **2000**, *30*, 451–462. [CrossRef]
28. Fang, Z.; Gao, X.; Zhang, H.; Tang, J.; Gao, Q. Application Layer Protocol Identification Method Based on ResNet. *Algorithms* **2025**, *18*, 52. [CrossRef]
29. Chivilikhin, D.; Ulyantsev, V.; Shalyto, A. Extended finite-state machine inference with parallel ant colony based algorithms. In Proceedings of the 7th International Student Workshop on Bioinspired Optimization Methods and Their Applications (BIOMA'14), Ljubljana, Slovenia, 9–10 October 2014; Jozef Stefan Institute: Ljubljana, Slovenia, 2014; pp. 117–126.
30. Pado, U.; Knebusch, A.; Mehmedovski, K. Computer-Based Methods for Adaptive Teaching and Learning. In *Advanced Technologies and the University of the Future*; Springer: Berlin/Heidelberg, Germany, 2024; pp. 297–317.
31. Demir, B.; Persello, C.; Bruzzone, L. Batch-mode active-learning methods for the interactive classification of remote sensing images. *IEEE Trans. Geosci. Remote Sens.* **2010**, *49*, 1014–1031. [CrossRef]
32. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; MIT Press: Cambridge, MA, USA, 1998; Volume 1.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.